University of Canberra
Faculty of Science and Technology

Software Technology 1
(4483/8995)
Week 6 Tutorial & Computer Lab
Assignment 2 Case Studies with Programming Problems
Case Study 3: Keras package and Google Colab

- This lab activity is assessed and contributes to the Assignment 2 mark.
- Please go through this Week_6_Tutorial_and Computer_Laboratory.doc available in Module 6 in Canvas
- For assessment of this activity, you need to complete the following in week 6:

  1) attend your allocated computer lab class in Week 6,

  2) complete all tasks listed in this handout,

  3) demonstrate the working activities to your tutor in class, and

  4) Save your python project to a zip file and upload this zip file to **In-class Lab Work Demo Week 6** and upload it to your GitHub Repository. Make sure you include a commit message with date and time, when you upload this zip file in GitHub.

- Your mark is 0 if you do not demonstrate your project.
- You can leave the class after you complete the 4 requirements above.

Tasks

- Introduction to Google Colab (https://colab.research.google.com/)
- Introduction to Matplotlib (https://matplotlib.org/)
- Introduction to Keras Library/Package (Keras: Deep Learning for humans)

Download the **ST1 Week 6 Tutorial and Computer Lab** word document in Canvas. This document contains all the necessary activities needed for each task completion in this week.

Task 1: Understanding Google Colab Programming Environment

- Go through the documentation for (https://www.marqo.ai/blog/getting-started-with-google-colab-a-beginners-guide) to understand what this environment can do.
- Matplotlib is already installed in Google colab.
- Keras package is already installed in Google colab.
- Matplotlib and Keras are two powerful packages in Python, similar to Pillow, Numpy and OpenCV.
- You can check the versions of matplotlib and Keras by entering the following commands in Google colab notebook cell

```
>> import matplotlib
>> import keras
>> print(matplotlib.__version__)
>> print(keras.__version__)
```

- **Note:**

After setting up Google colab, for each of the activity subtask questions, you need to write a python script and test it, or, test the python script (if the script is provided already) for each of the following questions in Task 1, and save the correctly working scripts as : task1_1.py, task1_2.py……..task1_7.py. The scripts (if not included) , the test results and/or screen shots need to be embedded in this document only and submitted to GitHub, after demonstration in the tutorial class to the tutor

Task 1_1: Loading an image and displaying it with Matplotlib and Keras. Replace the image here one of the with insect images from the Assets folder provided in the previous tutorial classes.

```
#loading an image
import matplotlib.pyplot as plt
# example of loading an image with the Keras API
from keras.preprocessing.image import load_img

# load the image
img_path =
'/content/drive/MyDrive/2026_Teaching/ST1/Week_6_Tu
tLab/bondi_beach.jpg'
img = load_img(img_path)

# report details about the image
print(type(img))
print(img.format)
print(img.mode)
print(img.size)

# show the image using matplotlib
plt.imshow(img)
plt.axis('off') # Turn off axis labels
plt.show()
```

- The expected console output for a successful test run for task1_1.py is as shown below:

Task 1_2: Convert the image to numpy array. Replace the image here with one of the insect images from the Assets folder provided in the previous tutorial classes

```
# example of converting an image with the Keras API
from keras.preprocessing.image import load_img
from keras.preprocessing.image import img_to_array
from keras.preprocessing.image import array_to_img
# load the image
img_path =
'/content/drive/MyDrive/2026_Teaching/ST1/Week_6_TutLab/bondi_bea
ch.jpg'
img = load_img(img_path)
print(type(img))
# convert to numpy array
img_array = img_to_array(img)
print(img_array.dtype)
print(img_array.shape)
# convert back to image
img_pil = array_to_img(img_array)
print(type(img))
```

- The expected console output for a successful test run for task1_2.py is as shown below:

```
<class 'PIL.JpegImagePlugin.JpegImageFile'>
float32
(427, 640, 3)
<class 'PIL.JpegImagePlugin.JpegImageFile'>
```

Task 1_3: Convert the image to grayscale, save the image, reload the image and display it. Replace the image here one of the with insect images from the Assets folder provided in the previous tutorial classes.

```
import matplotlib.pyplot as plt
import os
# example of saving an image with the Keras API
from keras.preprocessing.image import load_img
from keras.preprocessing.image import save_img
from keras.preprocessing.image import img_to_array
# load image as as grayscale
img_path =
'/content/drive/MyDrive/2026_Teaching/ST1/Week_6_TutLab/bondi_bea
ch.jpg'
img = load_img(img_path, color_mode='grayscale')
# convert image to a numpy array
img_array = img_to_array(img)
# save the image with a new filename
save_path_dir =
'/content/drive/MyDrive/2026_Teaching/ST1/Week_6_TutLab/'
save_filename = 'bondi_beach_grayscale.jpg'
full_save_path = os.path.join(save_path_dir, save_filename)
save_img(full_save_path, img_array)
# load the image to confirm it was saved correctly
img = load_img(full_save_path)
print(type(img))
print(img.format)
print(img.mode)
print(img.size)

# show the image using matplotlib
plt.imshow(img)
plt.axis('off') # Turn off axis labels
plt.show()
```

The expected console output for a successful test run for task1_3.py is as shown below:

Task 1_4: Loading a pre-existing image dataset (MNIST) from Keras datasets and summarizing it.

```
# load and summarize the MNIST dataset
from keras.datasets import mnist
# load dataset
(train_images, train_labels), (test_images,
test_labels) = mnist.load_data()
# summarize dataset shape
print('Train', train_images.shape,
train_labels.shape)
print('Test', (test_images.shape,
test_labels.shape))
# summarize pixel values
print('Train', train_images.min(),
train_images.max(), train_images.mean(),
train_images.std())
print('Train', test_images.min(),
test_images.max(), test_images.mean(),
test_images.std())
```

- The expected console output for a successful test run for task1_4.py is as shown below:

```
Downloading data from
https://storage.googleapis.com/tensorflow/tf-keras-
datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s
0us/step
Train (60000, 28, 28) (60000,)
Test ((10000, 28, 28), (10000,))
Train 0 255 33.318421449829934 78.56748998339798
Train 0 255 33.791224489795916 79.17246322228644
```

Task 1_5: Normalizing an image dataset (MNIST)

```
# example of normalizing an image dataset (MNIST)
from keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
# load dataset
(trainX, trainY), (testX, testY) = mnist.load_data()
# reshape dataset to have a single channel
width, height, channels = trainX.shape[1], trainX.shape[2], 1
trainX = trainX.reshape((trainX.shape[0], width, height, channels))
testX = testX.reshape((testX.shape[0], width, height, channels))
# confirm scale of pixels
print('Train min=%.3f, max=%.3f' % (trainX.min(), trainX.max()))
print('Test min=%.3f, max=%.3f' % (testX.min(), testX.max()))
# create generator (1.0/255.0 = 0.003921568627451)
datagen = ImageDataGenerator(rescale=1.0/255.0)
# Note: there is no need to fit the generator in this case
# prepare a iterators to scale images
train_iterator = datagen.flow(trainX, trainY, batch_size=64)
test_iterator = datagen.flow(testX, testY, batch_size=64)
print('Batches train=%d, test=%d' % (len(train_iterator),
len(test_iterator)))
# confirm the scaling works
batchX, batchy = next(train_iterator)
print('Batch shape=%s, min=%.3f, max=%.3f' % (batchX.shape,
batchX.min(), batchX.max()))
```

- The expected console output for a successful test run for task2_2.py is as shown below:

```
Train min=0.000, max=255.000
Test min=0.000, max=255.000
Batches train=938, test=157
Batch shape=(64, 28, 28, 1), min=0.000,
max=1.000
```

Task 1_6: Centering an image dataset (MNIST)

```
# example of centering a image dataset
from keras.datasets import mnist
from tensorflow.keras.preprocessing.image import
ImageDataGenerator
# load dataset
(trainX, trainy), (testX, testy) = mnist.load_data()
# reshape dataset to have a single channel
width, height, channels = trainX.shape[1], trainX.shape[2], 1
trainX = trainX.reshape((trainX.shape[0], width, height,
channels))
testX = testX.reshape((testX.shape[0], width, height,
channels))
# report per-image mean
print('Means train=%.3f, test=%.3f' % (trainX.mean(),
testX.mean()))
# create generator that centers pixel values
datagen = ImageDataGenerator(featurewise_center=True)
# calculate the mean on the training dataset
datagen.fit(trainX)
print('Data Generator Mean: %.3f' % datagen.mean.item())
# demonstrate effect on a single batch of samples
iterator = datagen.flow(trainX, trainy, batch_size=64)
# get a batch
batchX, batchy = next(iterator)
# mean pixel value in the batch
print(batchX.shape, batchX.mean())
# demonstrate effect on entire training dataset
iterator = datagen.flow(trainX, trainy,
batch_size=len(trainX), shuffle=False)
# get a batch
batchX, batchy = next(iterator)
# mean pixel value in the batch
print(batchX.shape, batchX.mean())
```

The expected console output for a successful test run for task1_6.py is as shown below:

```
Means train=33.318, test=33.791
Data Generator Mean: 33.318
(64, 28, 28, 1) -0.4113012
(60000, 28, 28, 1) -1.9512918e-05
```

Task 1_7: Standardizing an image dataset (MNIST)

.

```
# example of standardizing a image dataset
from keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
# load dataset
(trainX, trainy), (testX, testy) = mnist.load_data()
# reshape dataset to have a single channel
width, height, channels = trainX.shape[1], trainX.shape[2], 1
trainX = trainX.reshape((trainX.shape[0], width, height, channels))
testX = testX.reshape((testX.shape[0], width, height, channels))
# report pixel means and standard deviations
print('Statistics train=%.3f (%.3f), test=%.3f (%.3f)' % (trainX.mean(), trainX.std(),
testX.mean(), testX.std()))
# create generator that centers pixel values
datagen = ImageDataGenerator(featurewise_center=True,
featurewise_std_normalization=True)
# calculate the mean on the training dataset
datagen.fit(trainX)
print('Data Generator mean=%.3f, std=%.3f' % (datagen.mean.item(),
datagen.std.item()))
# demonstrate effect on a single batch of samples
iterator = datagen.flow(trainX, trainy, batch_size=64)
# get a batch
batchX, batchy = next(iterator)
# pixel stats in the batch
print(batchX.shape, batchX.mean(), batchX.std())
# demonstrate effect on entire training dataset
iterator = datagen.flow(trainX, trainy, batch_size=len(trainX), shuffle=False)
# get a batch
batchX, batchy = next(iterator)
# pixel stats in the batch
print(batchX.shape, batchX.mean(), batchX.std())
```

- The expected console output for a successful test run for task1_7.py is as shown

below:

```
Statistics train=33.318 (78.567), test=33.791 (79.172)
Data Generator mean=33.318, std=78.567
(64, 28, 28, 1) -0.016745089 0.98178655
(60000, 28, 28, 1) -3.4560264e-07 0.9999998
```